A

Project Report

on

# Selendroid-Based Mobile Application Testing in the DevOps Lifecycle

*Submitted for partial fulfilment of the requirements for the award of the degree of*

**BACHELOR OF TECHNOLOGY**

**IN**

**COMPUTER SCIENCE AND ENGINEERING**

**By**

**M.RASHMITHA**

**24K81A05P2**

Under the guidance of

**Mrs.M.DILEEP KUMAR**

**Assistant professor**




**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

## St. MARTIN'S ENGINEERING COLLEGE

**SHAPING VIKSIT BHARAT'S BRIGHTEST MINDS**

UGC Autonomous

Affiliated to JNTUH, Approved by AICTE

Accredited by NBA & NAAC A+,ISO 9001-2008 Certified

Dhulapally,Secunderabad-500 100

www.smec.ac.in

# St. MARTIN'S ENGINEERING COLLEGE

**SHAPING VIKSIT BHARAT'S BRIGHTEST MINDS**

UGC Autonomous

NBA& NAAC A+ Accredited

Dhulapally, Secunderabad - 500 100

www.smec.ac.in

## Certificate

This is to certify that the project entitled "**Selendroid-Based Mobile Application Testing in the DevOps Lifecycle**" is being submitted by **M.RASHMITHA-24K81A05P2** in fulfilment of the requirement for the award of degree of **BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING** is recorded of Bonafide work carried out by them. The result embodied in this report have been verified and found satisfactory.

**Project Internal Examiner**

**Mr.M.DILEEP KUMAR**

Assistant professor

Department of CSE

**Signature of HOD**

**Mr. D. KRISHNA KISHORE**

Associate Professor and HOD

Department of CSE

**Internal Examiner**

**External Examiner**

# St. MARTIN'S ENGINEERING COLLEGE

**SHAPING VIKSIT BHARAT'S BRIGHTEST MINDS**

UGC Autonomous

NBA& NAAC A+ Accredited

Dhulapally, Secunderabad - 500 100

www.smec.ac.in

## COMPUTER SCIENCE AND ENGINEERING

## DECLARATION

I, the student of "**Bachelor of Technology in Department of Computer Science and Engineering"**, session: 2024-2028, **St. Martin's Engineering College, Dhulapally, Kompally, Secunderabad** , here by declare that the work presented in this project work is **Selendroid-Based Mobile Application Testing in the DevOps Lifecycle** is the outcome of our own Bonafide work and is correct to the best of our knowledge and this work has been undertaken taking care of Engineering Ethics. This result embodied in this project report has not been submitted in any university for award of any degree.

**M.RASHMITHA-24K81A05P2**

# ACKNOWLEDGEMENT

The satisfaction and euphoria that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible and whose encouragement and guidance have crowded our efforts with success.

First and foremost, I would like to express my deep sense of gratitude and indebtedness to our College Management for their kind support and permission to use the facilities available in the Institute.

I especially would like to express my deep sense of gratitude and indebtedness to **Dr. K. RAVINDRA,** Director, Principal, Professor St. Martin's Engineering College Dhulapally, for permitting me to undertake this project.

I am also thankful **Mr.D. KRISHNA KISHORE,** Head of the Department of Computer Science and Engineering, St. Martin's Engineering College, Dhulapally, Secunderabad, for his support and guidance throughout my project.

I would like to express my sincere gratitude,thanks and indebtedness to my project supervisor **Mr.M.DILEEP KUMAR** Assistant Professor, Department of Computer Science and Engineering, St. Martins Engineering College, Dhulapally, for his support and guidance throughout project.

Finally, I express thanks to all those who have helped me in successfully completing this project. Furthermore, I would like to thank my family and friends for their moral support and encouragement. I express thanks to all those who have helped us in successfully completing the project.

**M.RASHMITHA-24K81A05P2**

# CONTENTS


| | | |
|---|---|---|
| 1. | ABSTRACT | 1 |
| 2. | INTRODUCTION | 2 |
| 3. | LITERATURE SURVEY | 3 |
| 4. | SYSTEM ANALYSIS | 4 |
| 5. | SYSTEM REQUIREMENT | 5 |
| 6. | ALGORITHM | 6 |
| 7. | SYSTEM IMPLEMENTATION | 7-10 |
| 8. | OUTPUT | 11 |
| 9. | CONCLUSION | 12 |
| 10. | FUTURE ENHANCEMENT | 13 |
| 11. | REFERENCES | 14 |

# 1. ABSTRACT

In the modern software development environment, mobile applications are widely used in various fields, making software quality and reliability essential. DevOps is a software development approach that integrates development and operations to improve collaboration, automate processes, and deliver software faster. Automated testing plays an important role in the DevOps lifecycle by identifying defects early and ensuring the quality of applications before deployment.

This project, "Selendroid-Based Mobile Application Testing in the DevOps Lifecycle," focuses on automating the testing of Android mobile applications using Selendroid, an automation testing framework designed for Android applications. Selendroid allows testers to automate user interactions such as clicking buttons, entering text, navigating between screens, and verifying application behavior. By integrating Selendroid with DevOps tools such as Jenkins, the testing process can be incorporated into a Continuous Integration and Continuous Delivery (CI/CD) pipeline.

The proposed system aims to reduce manual testing efforts, improve testing accuracy, detect defects at an early stage, and accelerate the software release process. Automated test cases can be executed whenever application changes are introduced, helping developers identify issues quickly and maintain application quality. The project demonstrates how automated mobile application testing can support the DevOps lifecycle and improve software delivery efficiency.

# 2. INTRODUCTION

Mobile applications are an important part of modern computing. They are widely used for education, communication, shopping, banking, entertainment, and many other activities. As the use of mobile applications increases, testing is necessary to ensure that applications work correctly, provide a good user experience, and meet quality standards.

DevOps is a software development approach that combines Development and Operations to improve collaboration, automate processes, and deliver software faster. It includes practices such as Continuous Integration (CI), Continuous Delivery (CD), automated testing, and continuous monitoring. Automated testing helps identify errors early and reduces the effort required for repetitive manual testing.

Selendroid is an automation testing framework designed for Android mobile applications. It allows testers to simulate user actions such as clicking buttons, entering text, navigating between screens, and checking application responses. These features help verify whether an Android application behaves as expected.

**Objectives**

To automate user interactions in Android applications.

To execute test cases and identify application defects.

To reduce manual testing effort and improve software.

**Scope**

The scope of this project is limited to demonstrating automated testing of Android mobile applications using Selendroid within a DevOps workflow. It focuses on automating basic user interactions, verifying application functionality, and examining test results. Jenkins may be used to demonstrate the execution of automated tests as part of a continuous integration process.

# 3.LITERATURE SURVEY

The literature survey provides an overview of existing research and technologies related to mobile application testing, automated testing frameworks, and DevOps practices. Various tools and methods have been developed to improve software quality, reduce manual testing, and accelerate the software development process.

### 3.1 Mobile Application Testing

Mobile application testing is an important phase of software development. It is used to verify the functionality, usability, compatibility, and reliability of mobile applications. Traditional manual testing requires testers to perform the same operations repeatedly, which can consume time and increase the possibility of human errors.

### 3.2 Automation Testing Using Selendroid

Selendroid is an open-source automation testing framework designed for Android applications. It allows testers to interact with application elements and simulate user activities such as clicking buttons, entering text, and navigating between screens.

### 3.3 DevOps and Continuous Integration

DevOps integrates software development and IT operations to improve collaboration and software delivery. Continuous Integration involves regularly integrating code changes into a shared repository and building the application to detect errors early. Tools such as Jenkins can automate build and test processes.

### 3.4 Continuous Testing in the DevOps Lifecycle

Continuous testing involves executing automated tests throughout the software development lifecycle. It provides rapid feedback about application functionality whenever code changes are introduced. By incorporating automated test cases into the DevOps pipeline, organizations can reduce repetitive manual work, detect defects earlier, and improve the reliability of software releases.

### 3.5 Summary

The literature survey shows that automated mobile application testing and DevOps practices can improve software quality and delivery efficiency. Selendroid provides a way to demonstrate automated Android testing, while Jenkins can support the automation of build and testing activities. Combining these concepts provides a foundation for understanding continuous testing within the DevOps lifecycle

# 4. SYSTEM ANALYSIS

## 4.1 Existing System

In the existing system, Android mobile applications are often tested manually by testers. Testers perform operations such as clicking buttons, entering text, navigating between screens, and checking application responses. These activities must be repeated whenever changes are made to the application. Manual testing can be time-consuming and may delay the identification of defects.

## 4.2 Limitations of Existing System

- Testing requires considerable manual effort and time.
- Repeated test execution is inconvenient and inefficient.
- Human errors may occur during testing.
- Defects may be identified late in the development lifecycle.

## 4.3 Proposed System

The proposed system uses Selendroid to automate the testing of Android mobile applications. Test scripts are created to perform user actions such as clicking buttons, entering text, navigating between screens, and verifying application behavior. The system executes these test cases automatically, reducing the need for repeated manual testing.

## 4.4 Feasibility Study

The testing process can be integrated with DevOps tools such as Jenkins to execute automated tests during the continuous integration process. Test results help developers identify errors and improve application quality before deployment. This approach aims to reduce testing time, improve consistency, and support faster software delivery.

# 5. SYSTEM REQUIREMENTS

**Hardware Requirements**

A desktop computer or laptop with a dual-core processor or higher, at least 4 GB RAM (8 GB recommended), and sufficient free disk space for the project files, Java development tools, Android SDK, and testing environment is required.

**Software Requirements**

- Operating System: Windows, Linux, or macOS.
- Languages: Java.
- Code Editor: Visual Studio Code, Notepad++, or another text editor.
- Browser: Google Chrome, Microsoft Edge, Mozilla Firefox, or another modern browser.
- Developer Tools: Eclipse IDE, IntelliJ IDEA, or another Java-compatible editor.

**Functional Requirements**

- The application shall accept task text from the user.
- The system shall connect to an Android application for testing.
- The system shall execute automated test scripts using Selendroid.
- The system shall simulate user actions such as clicking buttons and entering text.
- The system shall verify the expected behavior of application features.

**Non-Functional Requirements**

- Usability: controls and messages should be clear.
- Performance: task operations should respond quickly for a small list.
- Compatibility: the page should work in a modern web browser.
- Maintainability: HTML, CSS, and JavaScript should be kept in separate files.
- Reliability: valid actions should update the correct task.

# 6. ALGORITHM AND FLOW

**Algorithm**

- Start the testing process.
- Initialize the Selendroid testing environment and required dependencies.
- Launch the Selendroid server and connect the Android application or emulator.
- Identify the application elements to be tested.
- Execute automated test cases using Java test scripts.
- Simulate user actions such as clicking buttons, entering text, and navigating between screens.
- Verify the actual application output against the expected results.
- If the test passes, record the successful result.
- If the test fails, report the error and identify the cause.
- If Jenkins integration is configured, execute the test scripts through the CI pipeline.
- Review the test results and correct any identified defects.
- Repeat testing when necessary.
- Stop the process after completing the test cases.

**Flow Description**

Start → Configure testing environment → Launch Selendroid server → Connect Android application → Execute automated test cases → Verify application behavior → Are the test results successful? If yes, record successful results. If no, report errors and fix defects → Review test results → Repeat testing if necessary → Stop..

## 7. SYSTEM IMPLEMENTATION

```
import os, zipfile, textwrap

base = "/mnt/data/selendroid-devops-project"
os.makedirs(base + "/src/test/java/com/example", exist_ok=True)

files = {
"pom.xml": """<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-
4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>selendroid-devops-project</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>8</maven.compiler.source>
        <maven.compiler.target>8</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

     <dependencies>
         <dependency>
             <groupId>io.selendroid</groupId>
             <artifactId>selendroid-client</artifactId>
             <version>0.17.0</version>
         </dependency>
         <dependency>
             <groupId>junit</groupId>
             <artifactId>junit</artifactId>
             <version>4.13.2</version>
             <scope>test</scope>
         </dependency>
     </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>2.22.2</version>
            </plugin>
        </plugins>
    </build>
</project>
""",
"src/test/java/com/example/SelendroidTest.java": """package com.example;
```

```
import io.selendroid.SelendroidCapabilities;
import io.selendroid.SelendroidDriver;
import org.junit.After;
import org.junit.Before;
import org.junit.Test;
import org.openqa.selenium.By;
import org.openqa.selenium.WebElement;

import static org.junit.Assert.assertNotNull;
import static org.junit.Assert.assertTrue;

/**
 * Example automated test for the official Selendroid sample app.
 * Requires the compatible Selendroid server and sample APK to be installed
 * and running in a legacy-compatible Android test environment.
 */
public class SelendroidTest {
    private SelendroidDriver driver;

    @Before
    public void setUp() throws Exception {
        SelendroidCapabilities capabilities =
                new SelendroidCapabilities("io.selendroid.testapp:0.17.0");
        driver = new SelendroidDriver(capabilities);
    }

    @Test
    public void testTextFieldAcceptsInput() {
        WebElement textField = driver.findElement(By.id("my_text_field"));
        assertNotNull("Sample app text field should be present", textField);

        textField.clear();
        textField.sendKeys("Selendroid DevOps");

        String enteredText = textField.getAttribute("value");
        if (enteredText == null || enteredText.isEmpty()) {
            enteredText = textField.getText();
        }

        assertTrue(
                "Expected text to be entered into the sample app field",
                enteredText != null && enteredText.contains("Selendroid DevOps")
        );
    }

    @After
    public void tearDown() {
        if (driver != null) {
            driver.quit();
        }
    }
}
```

```
""",
"Jenkinsfile": """pipeline {
    agent any

    tools {
        // Configure a JDK named "JDK8" in Jenkins Global Tool Configuration.
        jdk 'JDK8'
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Build and Test') {
            steps {
                // Requires a compatible Selendroid server + Android test device/emulator
                // already configured on the Jenkins agent.
                sh 'mvn -B clean test'
            }
        }
    }

    post {
        always {
            junit allowEmptyResults: true, testResults: 'target/surefire-reports/*.xml'
        }
    }
}
""",
".gitignore": """target/
.idea/
.project
.classpath
.settings/
.DS_Store
""",
"README.md": """# Selendroid-Based Mobile Application Testing in the DevOps Lifecycle

## Project contents
- `pom.xml` — Maven dependencies and test configuration
- `src/test/java/com/example/SelendroidTest.java` — automated Android test
- `Jenkinsfile` — sample Jenkins pipeline

## Important compatibility note
Selendroid is a legacy Android automation framework. This project uses the Selendroid 0.17.0 client and the official Selendroid sample application identifier. It is not expected to work with every current Android SDK, device, JDK, or browser/driver combination. Use a compatible legacy test environment.

## Prerequisites
```

1. Java 8 and Maven.
2. Android SDK and a compatible emulator/device.
3. The official Selendroid 0.17.0 standalone server and sample test APK.
4. The sample app must be installed/available to the Selendroid server.

Get the server and sample app from the Selendroid project's setup/documentation page:
https://selendroid.github.io/selendroid.io/setup.html

## Run locally
1. Start an Android emulator/device supported by your selected legacy environment.
2. Start the Selendroid server using the downloaded standalone JAR and sample APK, following the matching version's documentation.
3. From this project directory, run:

   `mvn clean test`

## Jenkins
- Configure a Jenkins agent with Java 8, Maven, Android SDK, a compatible emulator/device, and a running/configurable Selendroid server.
- Configure a JDK tool named `JDK8` in Jenkins.
- Commit this project to a source-control repository and create a Pipeline job using the included `Jenkinsfile`.

## Test behavior
The test opens the Selendroid sample application, locates `my_text_field`, types `Selendroid DevOps`, and checks that the field contains the entered text.

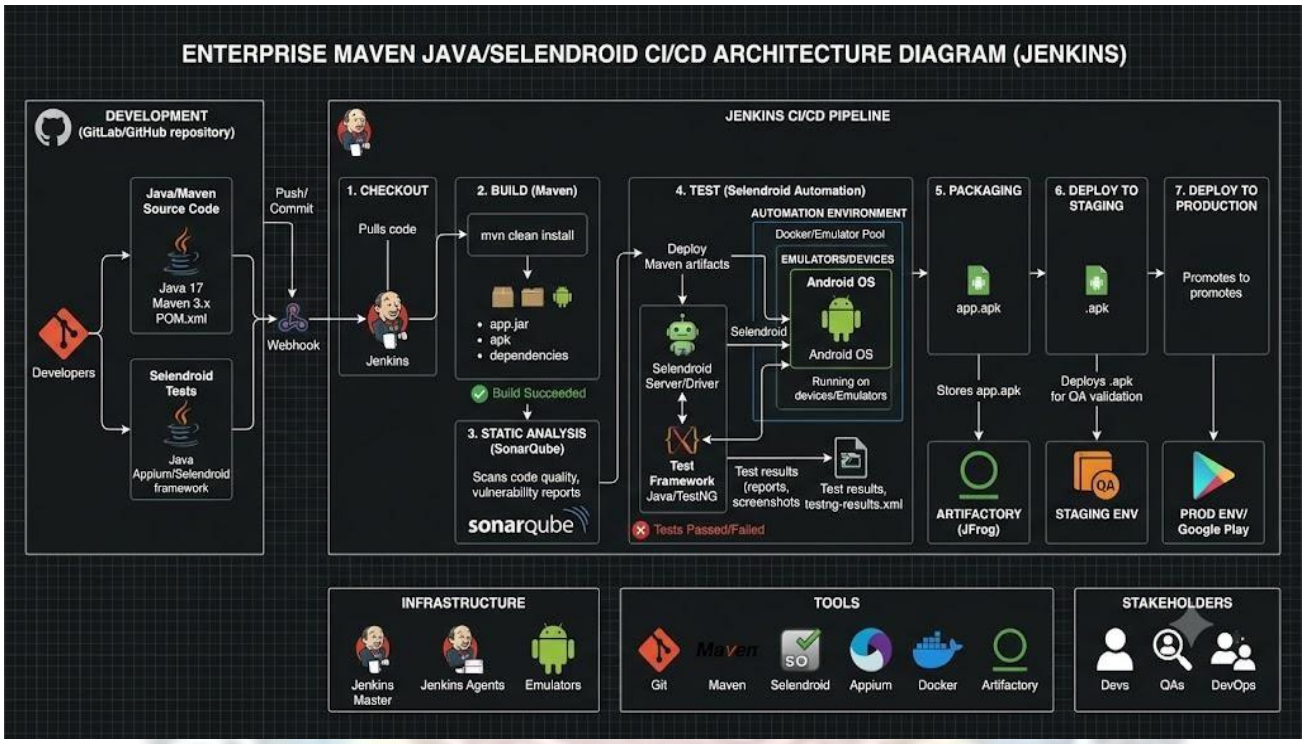
If the test cannot find `my_text_field`, verify that the official sample APK/version is being used and inspect the app's actual element IDs.
"""
}

```
for rel, content in files.items():
    path = os.path.join(base, rel)
    os.makedirs(os.path.dirname(path), exist_ok=True)
    with open(path, "w", encoding="utf-8") as f:
        f.write(content)

zip_path = "/mnt/data/selendroid-devops-project.zip"
with zipfile.ZipFile(zip_path, "w", zipfile.ZIP_DEFLATED) as z:
    for root, dirs, filenames in os.walk(base):
        for filename in filenames:
            full = os.path.join(root, filename)
            arc = os.path.relpath(full, os.path.dirname(base))
            z.write(full, arc)

print(f"Created project ZIP: {zip_path}")
print("Included files:")
for rel in files:
    print("-", rel)
```

# 8. OUTPUTS




TECHNOLOGY FOR PROSPERITY
UGC AUTONOMOUS

# 9. CONCLUSION

The project "**JavaScript**" demonstrates how JavaScript can be used with HTML and CSS to create a responsive and interactive web page. The task-list application allows users to add tasks, mark them as completed, and delete them. It also validates input and provides feedback when the user attempts to add an empty task.

The implementation provides practical experience with JavaScript variables, functions, conditional statements, event listeners, DOM manipulation, and element creation. Separating the HTML, CSS, and JavaScript files makes the application easier to maintain and understand. The use of textContent for task labels also helps ensure that user-entered content is treated as text.

The project is suitable for beginners because it uses a small number of files and can be executed directly in a modern browser. It demonstrates the importance of JavaScript in building useful client-side features. Additional capabilities such as saving tasks, filtering tasks, and connecting to a backend can be implemented in future versions.

TECHNOLOGY FOR PROSPERITY
UGC AUTONOMOUS

## 10.FUTURE ENHANCEMENTS

The Selendroid-Based Mobile Application Testing system can be improved in several ways to increase testing efficiency, application quality, and automation capabilities. The following enhancements can be considered for future development.

1. Integration with Advanced DevOps Tools: The system can be integrated with tools such as Jenkins, Git, and Maven to automate the build, testing, and continuous integration processes.
2. Cloud-Based Testing: The testing framework can be extended to cloud-based platforms to test applications on multiple Android devices and configurations without requiring physical devices for every test.
3. Cross-Device Compatibility Testing: Future versions can support testing across different screen sizes, device configurations, and Android versions to improve application compatibility.
4. Automated Test Report Generation: The system can generate detailed test reports containing passed test cases, failed test cases, execution time, and error details.
5. Parallel Test Execution: Multiple test cases can be executed simultaneously on different devices or emulators to reduce the total testing time.
6. Improved Error Detection: The system can be enhanced to capture screenshots, logs, and error messages when a test fails, helping developers identify and resolve problems more quickly.
7. Continuous Testing and Deployment: Automated tests can be executed whenever new code is committed. Successful builds can then proceed to later pipeline stages, subject to the project's deployment rules.
8. Migration to Modern Testing Frameworks: Since Selendroid is a legacy framework, the project can be upgraded to modern tools such as Appium to improve compatibility with newer Android versions and current development environments.

UGC AUTONOMOUS

# 11. REFERENCES


- 1. Selendroid Official Documentation. Selendroid: Selenium for Android Applications. Available at: https://selendroid.io/
- 2. Selenium Documentation. Selenium WebDriver Documentation. Available at: https://www.selenium.dev/documentation/
- 3. Android Developers. Android Developer Documentation. Available at: https://developer.android.com/
- 4. Apache Maven Project. Maven Documentation. Available at: https://maven.apache.org/guides/
- 5. JUnit Team. JUnit 4 Documentation. Available at: https://junit.org/junit4/
- 6. Jenkins Project. Jenkins User Documentation. Available at: https://www.jenkins.io/doc/
- 7. Git Documentation. Git Version Control System. Available at: https://git-scm.com/doc
- 8. Fowler, M. (2006). Continuous Integration. Available at: https://martinfowler.com/articles/continuousIntegration.html
- 9. Humble, J., and Farley, D. (2010). Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley Professional.
- 10. Appium Documentation. Appium: Cross-Platform Mobile Automation. Available at: https://appium.io/docs/en/latest/


TECHNOLOGY FOR PROSPERITY
UGC AUTONOMOUS